Scientific Report: Distributed System for Real-Time Analytics

Ungureanu Ana-Maria, Stoinea Maria-Miruna, Popoiu Andra

April 27, 2026

Contents

1 Introduction and Project Context

This report details the design and implementation of a distributed real-time analytics dashboard. The system was developed to monitor user interactions within a cinematographic platform as part of the Distributed Systems Design course. To ensure scalability, the entire architecture was built using Cloud Native technologies provided by the Google Cloud Platform (GCP) ecosystem.

The main challenge we addressed was the collection and processing of "view" events without impacting the performance of the primary application. Our solution uses an Event-Driven Architecture (EDA) to decouple the transactional flow, such as movie playback, from the analytical side responsible for statistical counting.

2 System Architecture and Components

Our system is built from a set of independent services designed to process analytical data without impacting the primary application's performance. By adopting an Event-Driven Architecture (EDA), we ensured that the user experience remains smooth and non-blocking.

2.1 Component Diagram

The high-level interaction between the distributed services and the data flow through the cloud ecosystem are illustrated in Figure 1. This diagram shows how events move from the initial user request to the final real-time update on the dashboard.

2.2 Service Description and Responsibilities

We have chosen to use a combination of managed cloud services to ensure the scalability and resilience of the project:

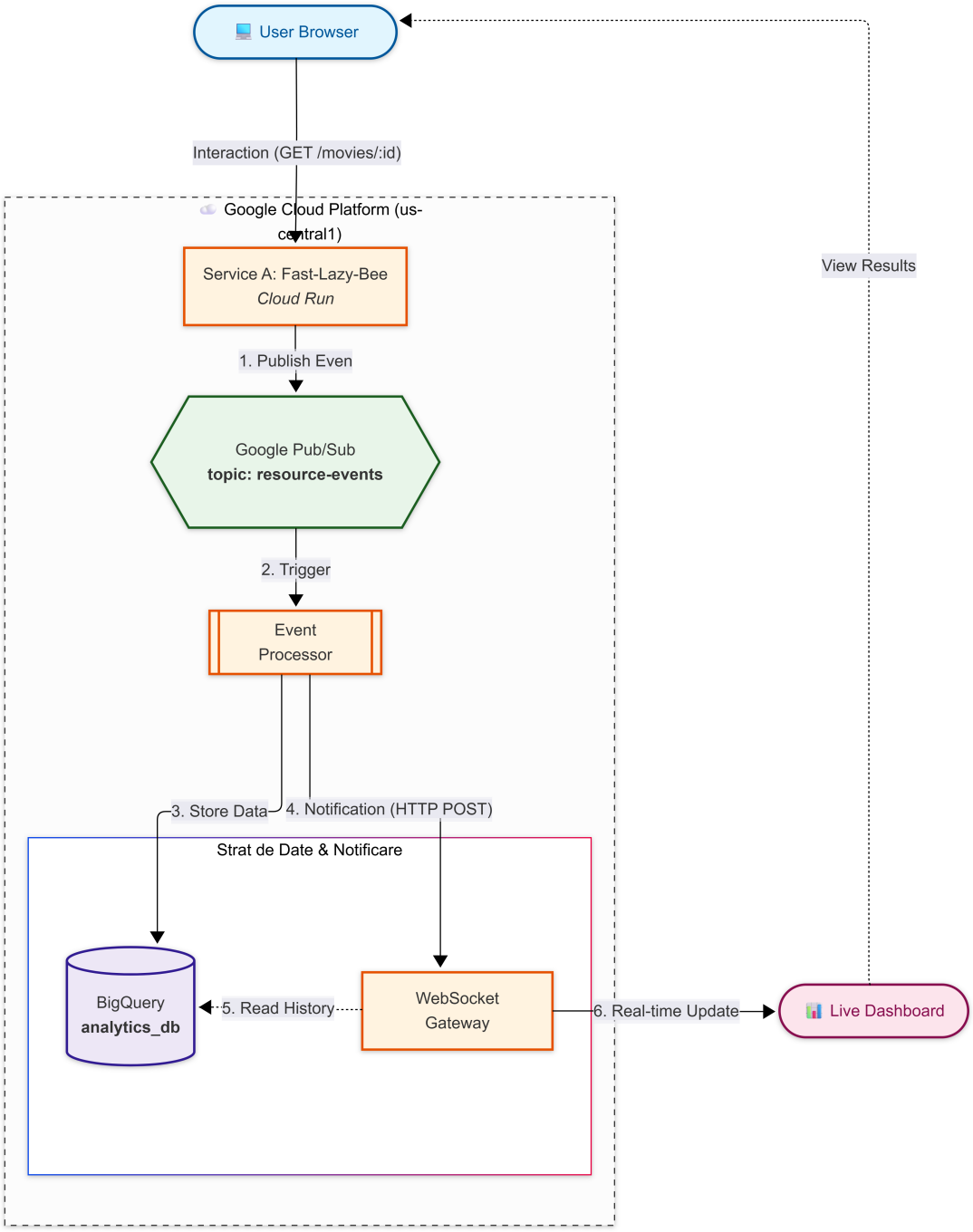

Figure 1: System Component Diagram

Component	Technology & Deployment	Analytical Responsibility
Service A (Fast-Lazy-Bee)	Node.js / Cloud Run (CaaS)	Data source; Publishes events to Pub/Sub on every view.
Event Processor	Cloud Functions (FaaS)	Consumes messages, implements idempotency, and writes to BigQuery.
Analytics Store	Google BigQuery	Analytical database (Stateful); Stores view history.
WebSocket Gateway	Node.js / Cloud Run (CaaS)	Fetches historical stats from BigQuery on connect and ensures real-time data push to clients.
Client Dashboard	HTML / Vanilla JS (Frontend)	Visualizes real-time analytics and provides the end-user interface for statistics.

Table 1: System Components and Responsibilities

2.3 Implementation Details and Data Flows

To understand how our system functions from a technical perspective, we have detailed below the responsibilities of each individual component. We will follow the data path step-by-step through the system, from event generation in the core application to real-time visualization, highlighting the implementation decisions that enable this flow.

2.3.1 Service A: Primary Application (Fast-Lazy-Bee)

As the foundation of our system, we utilized **Fast-Lazy-Bee**, an open-source movie API built with Fastify and TypeScript. This service handles movie metadata and user authentication using a MongoDB database. Our team containerized the application and deployed it on Google Cloud Run to ensure scalability.

The primary architectural modification was transforming this API into an active **event source** by performing the following tasks:

- **Event Generation:** We extended the movie routes to generate a unique event code using UUID v4 for every resource access.
- **Asynchronous Publishing:** We integrated the Google Cloud Pub/Sub library to push viewing events to the `resource-events` topic.
- **Latency Optimization:** The service responds to the user with a `200 OK` status immediately after publishing, ensuring the analytics pipeline does not slow down the browsing experience.

2.3.2 Google Pub/Sub: Inter-service Connectivity

We utilized Google Pub/Sub as a temporary storage layer between the application and the data processing stage. It was chosen because it acts as a buffer: if the processing service is busy or temporarily unavailable, messages are not lost but remain in the queue until they can be persisted. This model allows us to handle traffic spikes without blocking the system or losing critical information.

2.3.3 Cloud Functions: Automated Data Processing

To persist data into the analytical database, we developed a serverless function in Node.js 20. We opted for Cloud Functions due to its flexibility: the code only executes when a new message appears in Pub/Sub, thus optimizing resource consumption.

The function performs three essential tasks:

- **Decoding:** It takes the message from Base64 format and converts it back into a JSON object.
- **Data Preparation:** It formats the information according to the BigQuery schema (resource ID, timestamp, event ID).
- **Deduplication:** We use the `insertId` property (based on the unique ID generated at the source) to ensure that the same view is not counted twice, even if the network retransmits the message by mistake.

2.3.4 BigQuery: Statistics Storage

For long-term storage, we configured a dataset in BigQuery named `analytics_db`. Unlike a standard database, BigQuery allows us to perform calculations on large volumes of data extremely fast, serving as the centerpiece for our dashboard. We created the `view_stats` table with a clear structure to ensure all collected data is accurate and easily searchable.

2.3.5 WebSocket Gateway: Live Dashboard Updates

The final piece of the project is the WebSocket service, which also runs on Cloud Run. We applied specific configurations in the YAML file: we enabled *Session Affinity* to prevent user disconnections and increased the timeout to one hour.

The gateway performs two critical functions:

- **Startup History:** When the dashboard is opened, it queries BigQuery for the current status and immediately displays the latest views.
- **Real-time Updates:** The service has an endpoint named `/notify`. As soon as the processing function writes to BigQuery, it notifies the gateway, which instantly pushes the new data to all active dashboard users.

2.3.6 Client Dashboard: Real-time Visualization Interface

The frontend component serves as the final destination of our analytical pipeline, providing users with a near real-time overview of movie viewing activity. In line with the project requirements for a lightweight interface, we implemented the dashboard using standard HTML5 and Vanilla JavaScript, avoiding the overhead of heavy frameworks.

The dashboard's operation is strictly tied to the WebSocket Gateway:

- **Dynamic Connection:** Upon loading, the client establishes a persistent bi-directional connection using the `socket.io-client` library. This "pipe" remains open for up to 3600 seconds, as configured in our Cloud Run environment.
- **Bootstrap Data:** To prevent the dashboard from starting "empty", the client immediately receives a historical snapshot from BigQuery (retrieved via the Gateway) as soon as the connection is established.
- **Event-Driven Updates:** The dashboard does not perform periodic polling. Instead, it remains passive until it receives a `message` event from the Gateway. This event triggers an instantaneous update of the UI elements, such as view counts and recently accessed movies.

By offloading the heavy analytical queries to BigQuery and receiving only granular updates via WebSockets, we ensured that the client-side rendering latency remains negligible (milliseconds), making the overall propagation delay dependent almost exclusively on the backend cloud architecture.

3 Analysis of Communication in the Distributed System

The choice of how our services transmit data was one of the most critical design decisions. We combined synchronous, asynchronous, and real-time communication to achieve a balance between user response speed and the reliability of analytical data.

3.1 Synchronous Communication (HTTP REST)

We employed this model for the direct interaction between the Browser and Service A.

Justification: When a user requests movie details, they require an instantaneous response. HTTP REST is ideal here as it follows a request-response model. The user sends a request and waits for the server to return data from MongoDB. If this communication were asynchronous, the user would be left with a blank page until the system processed the request, completely compromising the browsing experience.

3.2 Asynchronous Communication (Publish/Subscribe)

For the analytics flow, from Service A to the Event Processor (Cloud Function), we transitioned to an asynchronous model using Google Pub/Sub.

Justification: This is the centerpiece that allows us to avoid slowing down the primary application. As soon as Service A receives a view request, it pushes the event to Pub/Sub and responds to the user with a `200 OK` status.

Why Pub/Sub: We chose the asynchronous variant to ensure service decoupling. If the analytics service were synchronous and encountered an issue or high latency (e.g., due to a function cold start), the user would have to wait. Through Pub/Sub, events are securely stored and processed in the background. Even if the processor is busy, messages remain in the queue and are automatically retried, ensuring that no statistical data is lost.

3.3 Real-Time Communication (WebSockets)

For the monitoring dashboard, we decided to use WebSockets instead of traditional HTTP.

Justification: If we had used HTTP, the dashboard would have to poll the database every few seconds for new data, wasting resources and creating unnecessary traffic. WebSockets allow us to maintain a persistent "pipe." Thus, our gateway can push data to the dashboard the exact second a new view is saved in BigQuery. This is the most efficient method to achieve the "live analytics" effect.

3.4 Inter-service Notification (Webhook / Push)

A less visible but critical flow occurs between the Cloud Function and the WebSocket Gateway.

Justification: After the function finishes writing a row to BigQuery, it performs a synchronous HTTP POST call to the gateway's `/notify` endpoint. We chose this synchronous model between the two services because we require an immediate trigger. The gateway must know instantaneously that the data is ready to be forwarded to the user. This link bridges the transition from asynchronous background processing back to the real-time flow seen by the user.

4 Consistency Analysis and the CAP Theorem Application

In a complex distributed system such as this, managing data state represents one of the greatest challenges. To ensure the dashboard functions correctly without slowing down the primary platform, we had to analyze the trade-offs imposed by the CAP Theorem.

4.1 Choosing the Design Model: AP System

According to the CAP Theorem, a distributed system can simultaneously guarantee only two of the three properties: Consistency (C), Availability (A), and Partition Tolerance (P).

In the cloud environment we utilized (Google Cloud Platform), Partition Tolerance (P) is a mandatory condition, as we cannot control potential network failures between Google's data centers. Therefore, our real choice was between Consistency (C) and Availability (A).

We decided that our system should be an AP (Availability & Partition Tolerance) type. We chose to prioritize Availability over strict Consistency for a very practical reason: a user accessing a movie should not be blocked or wait any extra time just for our statistics to update. It is far more important for the site to be fast (Available) than for the analytics dashboard to be updated in the exact same millisecond.

4.2 Eventual Consistency Model

As a result of choosing the AP model, our system employs eventual consistency. This means that while the data may not be identical in all locations at the exact moment an event occurs, it will reach the same state (synchronize) within a very short timeframe.

In practice, we observed this phenomenon through the existence of a consistency window. The data path is as follows:

1. Service A sends the message to Pub/Sub.
2. The Cloud Function picks it up and sends it to BigQuery.
3. BigQuery uses a streaming buffer to accept data rapidly.

Due to this flow, on the live dashboard, a view might appear with a delay of a few hundred milliseconds or even 1-2 seconds. This is a characteristic of OLAP (Online Analytical Processing) systems, where databases are optimized for high-throughput writing, accepting the fact that data is not instantaneously visible for queries.

4.3 Guaranteeing Accuracy through Idempotency

Even though we accepted eventual consistency, we did not want to sacrifice data accuracy (i.e., counting the same view multiple times).

A major challenge in distributed systems using message queues, such as Google Pub/Sub, is the "at-least-once" delivery guarantee. This means that in case of network errors or timeouts, Pub/Sub may send the same message two or more times to ensure it has been received.

To resolve this issue, we implemented idempotency in the Cloud Function (`index.js`):

- **ID Generation:** When a resource is accessed in Service A (`movie-id-routes.ts`), we generate a unique code for each event using UUID v4.

- **Destination Verification:** When the Cloud Function sends data to BigQuery, it uses this UUID as the `insertId`.
- **Result:** BigQuery uses this identifier to detect duplicates. If it receives two messages with the same ID within the same timeframe, it will keep only the first and ignore the second. Thus, we ensure that while the system is asynchronous and eventually consistent, the final report data remains accurate.

5 Performance and Scalability

In this section, we evaluate the system's capacity to handle high data volumes and analyze how the serverless infrastructure responds to intense demands. We performed stress tests to validate the event-driven architecture and confirm that decoupling flows through Pub/Sub effectively eliminates bottlenecks in the primary application.

5.1 Load Testing and Throughput Analysis

To find the limits of our system, we used the `hey` tool to send many simultaneous requests to Service A. We wanted to see how many requests per second (RPS) the service could handle before slowing down.

Scenario	Concurrency	Total Requests	Requests/sec	Avg Latency
Low Load	10	200	59.80	160.4 ms
Medium Load	25	500	146.86	157.7 ms
High Load	50	1000	**285.69**	159.2 ms

Table 2: Throughput and Performance metrics from Service A stress tests

Analysis: As seen in Table 2, the throughput increased linearly, reaching **285.69 requests/sec**. The most important observation is that the average latency stayed almost the same ($\sim$160 ms). This proves that Service A is not affected by the analytics processing; it just sends the event to Pub/Sub and is immediately ready for the next request.

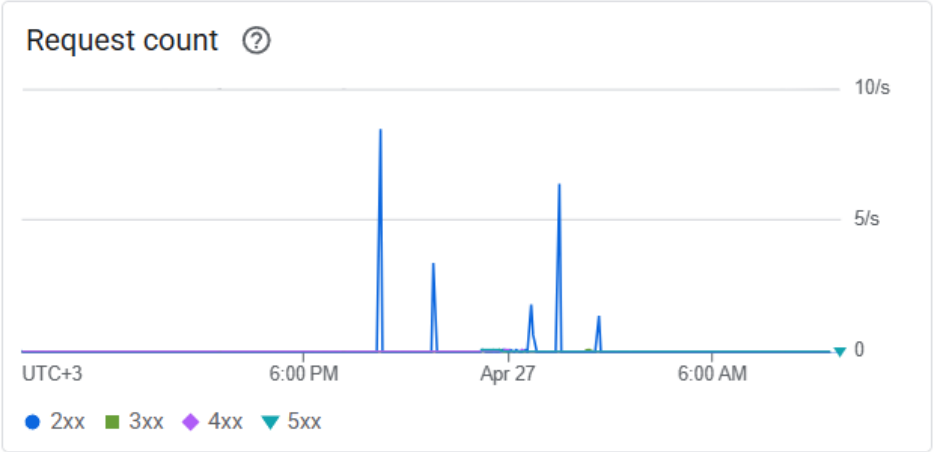

Figure 2: Traffic spikes during load testing captured via Google Cloud Monitoring.

5.2 Latency Analysis and Performance Profiling

To see how fast our system actually is, we ran a series of tests using a script that measures the time from the second a movie is viewed in Service A until it shows up on our dashboard. This "End-to-End" time tells us how long the data takes to travel through the whole cloud pipeline.

- **What we are measuring:** We found an average latency of **2032.25 ms**. We chose to measure the time until the data reaches the Gateway because the browser's time to display the info is almost zero and depends on your own laptop, not on our cloud setup. So, these 2 seconds are the "real" speed of our backend.
- **The Cold Start effect:** We noticed a big spike in the first test run, reaching **2801.6 ms**. This is a "Cold Start." It happens because Google Cloud needs a bit of extra time to start a new container when nobody has used the service for a while.
- **Warm Start (Normal Speed):** After the first few tries, the system became much faster. In the 5th run, the time dropped to **1014.58 ms**. This 1-second delay is the normal speed of our pipeline (Service A → Pub/Sub → Functions → BigQuery → Gateway) once all the cloud services are "awake" and running.

Test Iteration	Latency (ms)	System State
Run 1	2801.60	Cold Start (First run)
Run 2	1771.24	Warming up
Run 5	**1014.58**	Warm (Steady speed)

Table 3: Latency measurements from our test script

Analysis: The difference between the first run and the last one shows why we used `min-instances: 1` for Service A. It keeps the main app fast for users, but we can still see small delays when the background Cloud Functions need to scale up from zero to process the data.

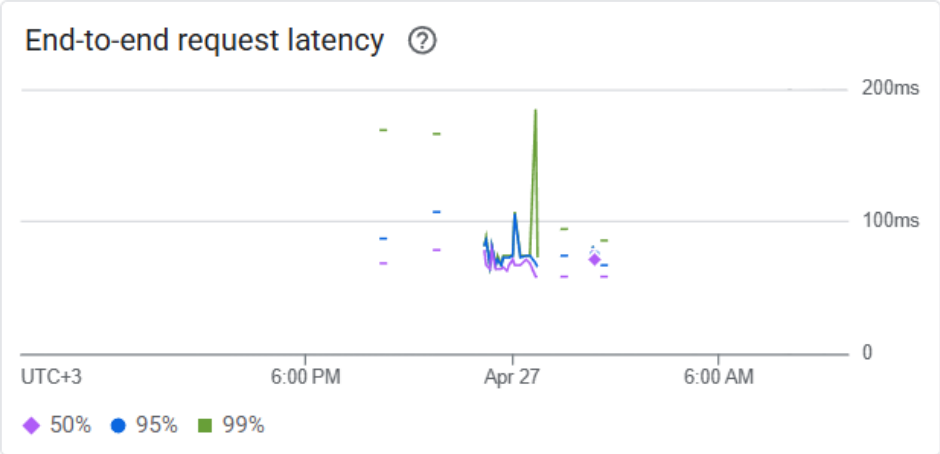

Figure 3: GCP Monitoring: Ingestion latency stability at the Service A level.

5.3 Consistency Window and CAP Theorem

Although Service A responds in milliseconds, it takes about **1.2 seconds** for the statistics to actually update on the dashboard. This is our "consistency window."

Practical Validation: This delay is exactly what we expected from our **AP (Availability and Partition Tolerance)** model. Service A stays available and fast, while the analytics part is "eventually consistent." The delay happens because:

- The message has to pass through Pub/Sub and the Cloud Function.
- **BigQuery Streaming Buffer:** BigQuery saves data quickly, but it takes a moment (about 1 second) before that data can be seen in a query.

5.4 Scalability Behavior Analysis

Our system's scaling is horizontal and automated. We configured Cloud Run for both Service A and the WebSocket Gateway to launch additional instances based on CPU utilization.

During load tests, we observed the following:

- **Service A Scaling:** As the number of requests increased, Cloud Run automatically started new instances. We set `min-instances: 1` specifically to ensure that the initial request is never delayed by a container cold start.
- **Cloud Functions Scaling:** This proved to be the most elastic component, scaling almost instantaneously for every message appearing in Pub/Sub.
- **Pub/Sub Management:** The message queue acted as a buffer, allowing Service A to operate at maximum speed even when the event processor required a few seconds to scale up.

5.5 Identification of Critical Bottlenecks

While the system scales effectively, we identified two specific areas where latency may occur:

- **BigQuery Streaming Buffer:** This is the most visible bottleneck. Even though data ingestion is fast, BigQuery requires a short window (the eventual consistency window) before data becomes available for querying. As a result, the dashboard updates with a slight delay rather than at a "microsecond" level.
- **Inter-service Network Latency:** Despite our efforts to keep all services within `us-central1`, the sequential communication steps (Pub/Sub → Cloud Functions → BigQuery → Gateway) introduce cumulative network latencies that affect the total processing time.

6 System Resilience

Resilience was a key goal for us. We wanted to make sure the dashboard keeps working even if there are network issues or if one service fails. To do this, we built several layers of protection to make sure no data is ever lost.

6.1 Fault Isolation through Asynchronous Design

The best way we ensured resilience was by using Google Pub/Sub as a middle layer. This creates "fault isolation," which means that if the analytics part or the BigQuery database stops working, Service A is not affected.

Real-world Impact: Users can keep watching movies without any lag or errors. Meanwhile, the view events are safely stored in the Pub/Sub queue, waiting for the other services to come back online.

6.2 Retry Policies and Data Persistence

For our Event Processor (Cloud Function), we turned on automatic retry policies provided by Google Cloud.

- **Retry Mechanism:** If the function fails to save data (for example, if BigQuery is busy), Pub/Sub will try to send the message again using an "exponential backoff" algorithm. This means it waits a bit longer between each try so it doesn't overwhelm the system.
- **Data Guarantee:** We kept the default 7-day retention period for messages. This gives us a full week to fix any code bugs without losing a single movie view event.
- **Handling Duplicates:** Because Pub/Sub might send the same message twice during a retry, we used the `eventId` as the BigQuery `insertId`. This ensures that even if a message is retried, it is only counted once in our stats.

6.3 Self-Healing and Dashboard Recovery

The WebSocket Gateway is the only part that stays "connected" to users, so it needs to recover fast if it restarts.

- **Automatic Recovery:** If the Cloud Run container crashes or restarts, it is programmed to immediately ask BigQuery for the latest history. This way, the dashboard is rebuilt instantly.
- **Auto-reconnection:** We used the `socket.io` library on the frontend. If the Gateway goes down for a few seconds, the browser will automatically reconnect as soon as it's back, and the user won't even have to refresh the page.

6.4 Infrastructure Security and IAM Permissions

During development, we realized that services couldn't talk to each other without the right permissions. We fixed this by setting up specific "Service Accounts" for each part of the system.

Our Approach: We followed the "Principle of Least Privilege." This means the Cloud Function only has permission to write to BigQuery, and the Gateway can only read from it. By keeping permissions limited, we made the system more secure—if one service has a problem, it can't accidentally affect the security of the whole project.

7 Comparative Analysis: Proposed System vs. Keystone Architecture (Netflix)

A crucial stage in our system analysis involved comparing our architecture with solutions employed by platforms managing massive data volumes. An industry benchmark is the Keystone pipeline developed by Netflix, designed to process hundreds of billions of events daily. Although the scale differs, the design principles applied in our project align with those used at a global level.

7.1 Data Ingestion: Pub/Sub vs. Apache Kafka

At Netflix, member interactions are transmitted by the API Gateway directly into Apache Kafka topics.

Correlation with the project: Similarly, we configured Service A to publish events to Google Pub/Sub.

Technical Objective: Both architectures avoid synchronous updates to the primary database. By utilizing Pub/Sub, we ensured that the analytical flow operates independently of the transactional flow, thereby preventing increased latency for the end-user.

7.2 Processing and Delivery Guarantees

Netflix utilizes processing engines such as Apache Flink to transform data from Kafka, placing a major emphasis on avoiding duplicates through idempotency tokens.

Our Implementation: We opted for a FaaS (Cloud Functions) solution, which offers the advantage of automated scaling based on the volume of incoming messages.

Integrity Guarantee: To handle "at-least-once" message delivery, we implemented idempotency using a unique `eventId` (UUID v4) generated in Service A, which acts as the `insertId` in BigQuery. This mechanism ensures statistical accuracy, eliminating the risk of a view being counted multiple times in the event of network redeliveries.

7.3 Storage and Data Consistency

While Netflix routes data to multiple destinations (Druid for rapid analytics or Hive for long-term storage), we centralized these functions within BigQuery.

Analytical Performance: BigQuery allows for data ingestion through its streaming buffer, providing support for the real-time queries required by our dashboard.

Consistency Model: Much like YouTube or Netflix, where view counts update at intervals rather than every single second for all users, our system employs an eventual consistency model. This is a direct consequence of prioritizing availability according to the CAP theorem, ensuring the service remains functional even during high-traffic periods.

7.4 Analysis Conclusion

This comparison confirms that the chosen architecture adheres to industry standards for data processing. Utilizing a message queue for service decoupling, implementing idempotency at the storage level, and accepting eventual consistency are technical decisions that align our project with the best practices of distributed systems engineering.

8 Conclusions and Future Work

The development of this real-time analytics system provided a comprehensive practical perspective on the challenges and advantages of cloud-native distributed architectures. By implementing an event-driven flow on Google Cloud Platform, we successfully achieved the project's primary objectives: total decoupling of services, automated scalability, and low-latency data visualization.

8.1 Key Outcomes and Technical Lessons

Throughout the implementation, several critical lessons regarding distributed systems were validated:

- **Architectural Decoupling:** Utilizing Google Pub/Sub proved to be the most effective decision for system stability. It allowed the transactional application (Service A) to remain highly responsive, regardless of any temporary spikes in analytics processing.
- **The CAP Trade-off:** We gained a practical understanding of the CAP theorem by observing that prioritizing Availability (A) and Partition Tolerance (P) inevitably leads to eventual consistency. This is a standard trade-off in high-scale systems where immediate consistency is less critical than service uptime.
- **Operational Resilience:** Implementing idempotency via unique event identifiers proved essential. In a distributed environment where "at-least-once" delivery is the norm, such mechanisms are the only way to ensure data integrity.

8.2 Project Limitations and Future Enhancements

While the current system is functional and performant, several directions for future development have been identified:

- **Advanced Analytics:** Integrating a processing engine like *Dataflow* (Apache Beam) would allow for complex windowing operations (e.g., calculating average watch time per session) before data reaches BigQuery.
- **Multi-region Deployment:** To further increase resilience, the services could be deployed across multiple GCP regions, using a Global Load Balancer to minimize latency for international users.
- **Monitoring and Alerting:** Implementing a centralized logging and alerting system via *Google Cloud Operations Suite* would allow for proactive identification of bottlenecks or failed function executions.

8.3 AI Transparency and Tool Usage

In accordance with the course requirements regarding the use of generative artificial intelligence, we transparently disclose that Large Language Models (LLMs), specifically **Google Gemini**, were utilized during the documentation phase of this project.

The AI assistant was primarily used for:

- **Text Reformulation and Translation:** Polishing technical descriptions and translating the draft content into a professional English format suitable for a scientific report.
- **LaTeX Structuring:** Assisting in the conversion of the report structure and component diagrams into LaTeX code compatible with the Overleaf environment.
- **Base Code and Script Generation:** Generating initial boilerplate code for the Cloud Function and WebSocket Gateway, as well as creating the PowerShell benchmarking scripts (`test-latency.ps1`, `test_consistency.ps1`, `test_recovery.ps1`) based on our architectural instructions.

All AI-generated content and suggestions were manually reviewed, verified for technical accuracy against our actual implementation, and adapted to ensure they accurately represent the experimental results and architectural decisions made by the team.

8.4 Final Remarks

In conclusion, the project demonstrates that modern cloud tools allow for the rapid construction of complex data pipelines that follow the same design principles as large-scale industrial solutions. This distributed approach not only improves current performance but also provides the necessary foundation for future growth and evolution of the platform.